

Project Proposal: Vectorized Loop Shape Optimization in MLIR

Muhammad Mazaya (mmazaya)

Justin Kim (yutarkk)

October 2025

1 URL for Project Web Page

[Project URL](#)

2 Project Description

We will build a loop–vector shape optimization pass in MLIR that lowers high-level `linalg` operations to the `vector` dialect while choosing logical vector sizes and remainder-handling strategies.

2.1 Motivation

When vectorizing loops, a simple approach is to always use the full hardware vector width and handle any remaining elements separately. While this approach is straightforward, it often performs poorly when trip counts are not multiples of the width, when memory alignment is irregular, or when predication overhead becomes significant. Rather than committing to one fixed width, this project makes vector shape decisions adaptive: choosing a logical vector size $L \leq W$ (where L is the logical vector size and W is the hardware vector size) and a tail-handling strategy such as masking, loop peeling, or scalar cleanup. These choices are then guided by a lightweight target-aware cost model that takes into account of architectural constraints (alignment, masking costs, etc.).

2.2 Core idea

Given an iteration domain and hardware width W , synthesize candidates of L and strategy $R \in \{\text{mask}, \text{peel}, \text{scalar}, \text{mixed}\}$. Estimate cost from instruction counts, mask/permute overhead, loop-control cost, alignment effects, and register pressure. Materialize the best plan as `scf.for` plus `vector.*` operations and lower to LLVM.

2.3 Deliverable

An MLIR pass that enumerates vector shapes, selects a plan via a cost model, emits `vector` IR with the chosen L and remainder strategy, with benchmarks to compare the performance of each of the strategies.

3 Goals

- 75% Goal: implement the `linalg` $\rightarrow$ `vector` pass with shape-enumeration hooks and IR templates; functional and tested on 1D (and selected 2D) ops with correct lowering.
- 100% Goal: implement multiple remainder-handling strategies with codegen, lowering, and tests.
- 125% Goal: implement and integrate a target-aware cost model to select among strategies and logical sizes with benchmarks.

4 Logistics

4.1 Plan of Attack and Schedule

1. 10/24–10/31: Study MLIR's pass infrastructure and the `linalg` and `vector` dialect APIs. Design the transformation strategy from `Linalg` to `Vector` and identify the subset of operations to support.

2. 11/1–11/7: Implement the initial vectorization pass that converts selected linalg operations into equivalent `vector.transfer` and `vector.contract` constructs.
3. 11/8–11/19: Implement lowering pass: legalize produced `vector` IR to LLVM, verify intrinsics mapping and canonicalization.
4. 11/20–11/27: Implement different shape strategies: masking, single-epilogue peel, mixed plan, and explore more strategies to optimize the vectorization shape.
5. 11/28–12/4: Implement cost model to pick the best shape for vectorization and benchmark the result.

4.2 Milestone

By November 20, we will have a functional Linalg→Vector prototype for a static shape, producing valid vector dialect IR suitable as a baseline for more strategies.

5 Literature Search

We have reviewed materials on MLIR vectorization and compiler design principles, including:

- Andrzej Warzyński and José Caballero, "Vectorization in MLIR," LLVM Developers' Meeting 2023 (slides).
- Andrzej Warzyński, "Vectorization in MLIR (Part II)," LLVM Developers' Meeting 2024 (slides).
- Renato Golin, "Auto Vectorization in LLVM," LLVM Developers' Meeting 2014 (slides).
- Nicolas Bohm Agostini et al., "AXI4MLIR: User-Driven Automatic Host Code Generation for Custom AXI-Based Accelerators," *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2024.

The 2023–2024 MLIR talks highlight the current reliance on hand-tuned tile sizes and acknowledge the need for cost-model-based automation, which remains an open problem. Golin’s earlier LLVM vectorizer work provides the multiple strategies to deal with vectorization within a loop in LLVM. The AXI4MLIR framework by Agostini et al. demonstrates how target-specific information can guide automatic code generation^x. Our work takes inspiration from these ideas to explore a similar cost-model-driven decision process within MLIR’s vectorization framework.

6 Resources Needed

Software: LLVM/MLIR, Clang/LLVM toolchain, CMake, Python utilities for scripting tests.

Hardware: AWS ARM instance or equivalent CPU supporting vector instructions.

7 Getting Started

We have begun reviewing the MLIR infrastructure and the linalg/vector dialect APIs, as well as prior vectorization examples. Next, we will define the initial operation set (linalg.matmul, elementwise ops) and prototype a minimal pass pipeline to exercise the transformation and testing harness.